

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar Automatizálási és Alkalmazott Informatikai Tanszék

ÖNÁLLÓ LABORATÓRIUM BESZÁMOLÓ

(BMEVIAUAL04)

Téma:

ReviewAgent

Alkalmazott MI, LLM, RAG és Agentic AI architektúrák
fejlesztése

Készítette:
Szabó Bálint

Konzulens:
Ekler Péter

2026. 05. 21.

1. Bevezetés, témaválasztás aktualitása

1.1. Bevezetés

Napjaink szoftverfejlesztési trendjeit radikálisan átformálják a mesterséges intelligencia (MI) alapú megoldások, különösen a nagy nyelvi modellek (Large Language Models – LLM) megjelenése és elterjedése. A generatív MI eszközök már nem csupán elszigetelt chat-asszisztensként funkcionálnak, hanem egyre inkább integrálódnak a vállalati alapfolyamatokba, értéknövelt szolgáltatásokat hozva létre.

Jelen dokumentáció tárgya a **ReviewAgent** nevű, felhőalapú, automatizált hírnévmenedzsment-rendszer tervezése, implementációja és kiértékelése. A szoftver célja, hogy a kis- és középvállalkozások számára automatizálja és optimalizálja a Google Business Profile online platformra beérkező ügyfélértékelések kezelését. A rendszer magját egy olyan hibrid architektúra alkotja, amely ötvözi a modern aszinkron backend-technológiákat, a felhőalapú relációs és vektor-adatbázisokat, valamint a technológia vezető Retrieval-Augmented Generation (RAG) és autonóm ágens (Agentic AI) tervezési mintákat.

1.2. A témaválasztás aktualitása és piaci kontextusa

A digitális korban egy vállalkozás online jelenléte és hírneve közvetlen hatással van annak gazdasági sikerére. A statisztikák szerint a fogyasztók több mint 90%-a rendszeresen olvas online értékeléseket, mielőtt szolgáltatást vagy terméket választana, és a Google értékelések bírnak a legnagyobb súllyal a keresőoptimalizálás - Local SEO - rangsorolási algoritmusában.

Ennek ellenére a KKV szektor szereplői komoly strukturális kihívásokkal néznek szembe ezen a területen, ezek közül kiemelném a leghangsúlyosabbakat alább:

Erőforrás-hiány: A beérkező vélemények monitorozása, elemzése és megválaszolása jelentős humán erőforrást és időt igényel, amelyet a mikrovállalkozások gyakran nem tudnak hatékonyanallokálni.

Reakcióidő kritikus szerepe: A platformok algoritmusai pozitívan értékelik a gyors válaszreakciót, míg az elhúzódó válaszadás csökkenti az ügyfélelégedettséget és rontja a digitális láthatóságot.

Kontextus- és stílushűség: Az automatizált, sablonalapú válaszok személytelenek, gyakran kontraproduktívak, és rombolják a márkába vetett bizalmat.

1.3 Pszichológia és megoldás

A fogyasztói magatartáskutatások és a digitális marketingpszichológia rávilágít egy kritikus anomáliára az online interakciók terén: a felhasználók részéről a véleményírás egy kétoldalú kommunikációs szándékot feltételez. Amikor egy ügyfél időt és energiát szán arra, hogy részletes szöveges értékelést fogalmazzon meg egy vállalkozásról, implicit módon elvárja a transzparens reakciót.

Amennyiben a Google Maps-en az látható, hogy a menedzsment szisztematikusan válasz nélkül hagyja a korábbi visszajelzéseket, a potenciális ügyfelekben a 'viszonzottság hiánya' (lack of reciprocity) elve lép életbe. A látogatók értéktelennek és egyoldalúnak érzékelik a platformot, ami egyenes úton vezet oda, hogy elállnak a véleményírási szándékuktól. Ez a jelenség súlyos következményekkel jár a KKV-k számára: a csökkenő véleményaktivitás miatt a Google algoritmus a vállalkozást inaktívnak minősíti, ami a keresési találati listákon való azonnali visszaesést, ezt hívjuk Local SEO degradációnak, és hosszú távon mérhető ügyfél- és bevételkiesést eredményez.

Bár a generatív nyelvi modellek, mint a Google Gemini vagy az OpenAI GPT modellek képesek lennének egyedi válaszok megfogalmazására, közvetlen, "out-of-the-box" alkalmazásuk vállalati környezetben kritikus kockázatokkal jár. A modellek belső tudására támaszkodó generálás hajlamos a **hallucinációra**, vagyis fiktív adatok, nem létező vállalati információk vagy szolgáltatások kitalálása, valamint képtelen figyelembe venni az adott cég egyedi üzleti logikáját, korábbi kommunikációs mintáit és specifikus stílusirányelveit.

A téma aktualitását pontosan ez a technológiai feszültség teremti meg: hogyan lehet egy alapvetően sztochasztikus működésű nyelvi modellt determinisztikus üzleti keretek közé szorítani?

A feladat megoldására a legmodernebb mérnöki megközelítést, a **RAG (Retrieval-Augmented Generation)** architektúrát alkalmazzuk. A RAG lényege, hogy az LLM számára a generálási fázis előtt egy külső, validált tudásbázisból, jelen esetben egy Qdrant vektoradatbázisból, keresünk releváns kontextust, így a modell nem a belső súlyaiból tippel, hanem a kapott tényadatok alapján fogalmaz. Ezt egészíti ki az **Agentic AI** megközelítés, ahol a rendszer nem egy statikus transzformációs pipeline, hanem egy autonóm ágensként viselkedik: elemzi a

bejövő hangulatot, mérlegeli a szabályrendszert, és eszközmeghívások, adatbázis-lekérdezések bevonásával önállóan hoz döntést a választervezet struktúrájáról.

A fejlesztés során így nem csupán egy elméleti modellt vizsgáltunk meg, hanem a Budapesti Műszaki és Gazdaságtudományi Egyetem témakiírásában szereplő elvárásoknak megfelelően napjaink technológiáit alkalmazva: FastAPI, PostgreSQL, Qdrant Cloud, Google Gemini SDK ötvözzük egy piackész, biztonságos, éles környezetben is futtatható SaaS (Software-as-a-Service) webalkalmazássá.

2. Feladat/problémakör bemutatása

2.1. A kiinduló állapot és a funkcionális specifikáció

A szoftver tervezési fázisát megelőzően részletes funkcionális specifikációt készítettem annak érdekében, hogy a végponti felhasználók, a kis- és középvállalkozások üzletvezetői, egy letisztult, technikai előismereteket nem igénylő, mégis robusztus felületen keresztül menedzselhessék a nekik írt véleményeket.

Tudtam, hogy már a fejlesztés korai szakaszában váltanom kell, egy nagyobb szinten, profibban kell kezelnem a projektet, így a funkcionális követelményeket az alábbi alapvető modulokra osztottam:

Autentikáció és többügyletes (Multi-tenant) környezet: A felhasználóknak biztonságos módon kell tudniuk regisztrálni és bejelentkezni a felületre. Mivel SaaS (Software-as-a-Service) modellről van szó, a rendszernek képesnek kell lennie több különálló vállalkozást vagy céget kiszolgálni úgy, hogy azok adatai transzparensen szeparáltak maradjanak.

Google Business Profile (GBP) integráció: A szoftvernek közvetlen vagy szimulált API-kapcsolaton keresztül szinkronizálnia kell az adott vállalkozáshoz beérkező Google-értékeléseket (szerző neve, csillagok száma, szöveges tartalom, egyedi belső azonosító).

Automatizált, kontextus- és stílusvezérelt válaszgenerálás: Ez a rendszer legfőbb funkcionális értéke. A beérkező új véleményre a szoftvernek automatikusan generálnia kell egy választervezetet. Ez a tervezet nem lehet általános sablon: alkalmazkodnia kell a cég egyedi kommunikációs irányelveihez – például közvetlen vagy formális stílus –, és kötelezően a cég valós adataira (nyitvatartás, szolgáltatások, árak) kell épülnie.

Irányítópult (Dashboard) és manuális felülbírálat: A generált válaszok nem publikálódhatnak azonnal kontroll nélkül. A felhasználónak egy interaktív felületen látnia kell a beérkezett véleményeket és az AI által felkínált választervezeteket, amelyeket jóváhagyhat, szabadon szerkeszthet vagy újrageneráltathat.

Üzleti analitika: A vezetők számára elengedhetetlen a statisztikai modul, amely vizuálisan grafikonok segítségével összesíti a cég értékeléseinek eloszlását, az átlagos pontszámot, valamint automatizált, aggregált vezetői jelentéseket készít a hangulatváltozásokról.

2.2. Nem-funkcionális követelmények és mérnöki kritériumok

A szoftver hosszú távú üzemeltethetősége, biztonsága és skálázhatósága érdekében a nem-funkcionális követelményeket szigorú iparági szabványok mentén határoztam meg:

Adatbiztonság és autorizáció: Az API-végpontok védelmét JWT (JSON Web Token) alapú architektúrával kell biztosítani. A kliensoldali biztonság érdekében a tokenek átadását és tárolását úgy kell megtervezni, hogy az ellenálljon a Cross-Site Scripting (XSS) és az Insecure Direct Object Reference (IDOR) típusú támadásoknak. Kiemelt követelmény, hogy egy bejelentkezett üzletvezető se manuálisan, se módosított API-hívásokon keresztül ne férhessen hozzá egy másik vállalkozás véleményeihez vagy belső adataihoz.

Alacsony válaszidő (Latency optimalizálás): Az LLM-alapú szöveggenerálás számításigényes folyamat. A nem-funkcionális elvárás az, hogy a felhasználói élmény ne sérüljön: a háttérfolyamatokat (adatbázis-szinkronizáció, vektorizálás) aszinkron módon, háttérfeladatként kell végrehajtani, míg a közvetlen UI-kiszolgálás válaszidejének 2 másodperc alatt kell maradnia felhőalapú modellek használata esetén.

Skálázhatóság: A rendszert úgy kell felépíteni, hogy felhőkörnyezetbe (amiben megépítettem először: GoHighApp, később: Render, AWS) könnyen telepíthető legyen, az adatbázisok képesek legyenek több tízezer rekordot és nagy dimenziójú vektoros beágyazásokat hatékonyan, minimális erőforrás-növekedéssel kezelni.

2.3. A generatív nyelvi modellek korlátai és technológiai kihívásai

A feladat megvalósítása során a legnagyobb mérnöki kihívást az LLM-ek (Large Book/Nyelvi modellek) belső természetéből fakadó korlátok jelentik. Bár egy alapmodell képes nyelvtanilag helyes, meggyőző magyar szöveget generálni, vállalati környezetben önmagában, úgynevezett "Zero-Shot" promptolással alkalmazhatatlan a következő okok miatt:

A hallucináció problémája: A nyelvi modellek statisztikai alapon jósolják meg a következő szót. Ha a modell nem ismeri az adott vállalkozás pontos belső szabályzatát vagy étlapját, hajlamos fiktív tényeket, nem létező telefonszámokat vagy kedvezményeket kitalálni a válaszban, ami jogi és üzleti kockázatot jelent.

A kontextus-ablak gazdaságos kezelése: A modellek bemeneti mérete korlátozott, és a token-alapú elszámolás miatt rendkívül gazdaságtalan lenne a cég összes korábbi adatát, leírását és ezer korábbi véleményét minden egyes promptba bemásolni.

A determinizmus hiánya: Ugyanarra a bemenetre a modellek képesek eltérő hangvételi vagy struktúrájú válaszokat adni. Biztosítani kell egy olyan szoftveres keretrendszert, amely mederben tartja a generálási folyamatot.

2.4. A Multi-tenant adatszeperáció problémája a vektortérben

A hagyományos relációs adatbázisokban mint a PostgreSQL a többügyletes működést viszonylag egyszerűen, egy `business_id` külső kulcs (Foreign Key) bevezetésével és megfelelő WHERE feltételek alkalmazásával biztosítani lehet.

A modern AI-alkalmazásokban használt **vektoradatbázisoknál** mint a Qdrant azonban ez a kérdés sokkal összetettebb. A RAG folyamat során a szövegeket egy embedding modell segítségével egy magas dimenziós - a Gemini esetében 3072 dimenziós - vektortérbe képezzük le. Ha a koszinusz-hasonlóság matematikai hasonlósági keresést globálisan futtatnánk le a teljes adatbázison, fennállna a veszélye annak, hogy az "A" étterem vendégének véleményére adott válaszhoz a rendszer a "B" fogászat belső adatait hívná le kontextusként, amennyiben a szövegek szemantikailag hasonlóak (pl. mindkettőben szerepel a "kedves kiszolgálás" vagy a "tiszta környezet" kifejezés).

A mérnöki feladat tehát egy olyan architektúra tervezése volt, amely képes a **Payload Filtering** technológiát alkalmazva a matematikai vektorkeresést szigorúan lekorlátozni az adott `business_id`-hoz tartozó pontokra. Így garantálható, hogy a különböző cégek adatai és egyedi stílusirányelvei soha, semmilyen körülmények között ne keveredhessenek össze a közös vektortérben, fenntartva a SaaS rendszerektől elvárt maximális adatbiztonságot.

3. Félév ütemezése, előzmények előző félévekről

3.1. Egyetemi előzmények és technológiai átállás

A ReviewAgent rendszer megtervezése és implementálása egy komplex, több rétegből álló mérnöki feladat volt, amelyhez a mérnökinformatikus alapképzés során elsajátított tárgyak biztosították a stabil elméleti és architektúráis alapokat.

Bár a hivatalos kötelező tanterv keretein belül a **Python** programozási nyelv oktatása nem szerepel – az egyetemi oktatás fókusza a magas szintű objektumorientált nyelvek (Java, C#) és a hardverközelibb megoldások (C, C++) felé hajlik –, a VIK-en oktatott alapelvek nyelvfüggetlenek és elengedhetetlenek voltak a sikerhez. A technológiai váltást és a Python aszinkron ökoszisztémájának (FastAPI) megismerését az alábbi korábbi tanulmányok tették lehetővé:

Szoftvertechnológiák: a szoftvertervezési mintákat, a többretegű (kliens-szerver) architektúrák felépítését, valamint a REST API-k tervezésének szigorú szabályait itt ismertem meg. Ez a tudás tette lehetővé a rendszer biztonságos és skálázható backendjének megtervezését, függetlenül attól, hogy a megvalósítás végül nem Java, hanem Python nyelven történt.

Adatbázisok: A relációs adatbázisok tervezése, a normalizálás, valamint az SQL nyelv mélyreható ismerete elengedhetetlen volt a felhasználók és a metaadatok szakszerű tárolásához a felhőalapú PostgreSQL adatbázisban.

Kliensoldali Rendszerek, IT Biztonság, Mesterséges Intelligencia, és még sok más...

3.2. A fejlesztés 12 hetes ütemterve (Mérőldkövek)

Mivel az alkalmazott technológiák jelentős része (RAG architektúra, vektorizálás, JWT-alapú Python biztonság) újdonságként hatott az egyetemi tananyaghoz képest, a félév során egy szigorúan strukturált, iteratív fejlesztési és kutatási ütemtervet kellett követnem.

A féléves munkafeladatokat egy 12 hetes ütemterv alapján osztottam fel, amely logikusan követte a szoftverfejlesztés életciklusát a piackutatástól a felhőbe történő telepítésig:

- **1. hét: Témafelvetés és szakirodalom-kutatás.** A KKV szektor online hírnévmenedzsment-problémáinak feltérképezése, az elérhető Nagy Nyelvi Modellek (LLM) és az Agentic AI koncepciók tanulmányozása.
- **2. hét: Nyelvi modellek lokális tesztelése.** Kísérletek futtatása nyílt forráskódú modellekkel (Először: Gemma29B) lokális hardveren, Docker és Ollama környezetben. A hardveres és válaszidő-beli korlátok azonosítása.

- **3. hét: Technológiaváltás és API integráció.** Döntés a felhőalapú megoldások mellett a teljesítménybeli akadályok (TTFT - Time To First Token) kiküszöbölése érdekében. A Google Gemini SDK (google-genai) integrációjának előkészítése.
- **4. hét: RAG koncepció kidolgozása.** A Retrieval-Augmented Generation logikai pipeline-jának megtervezése, az embedding modellek vagyis a 3072 dimenziós vektorok működésének vizsgálata.
- **5. hét: Vektoradatbázis (Qdrant) integrálása.** A Qdrant Cloud beállítása, a Multi-tenant adatszeparáció megtervezése payload alapú szűrések (Filter/FieldCondition) segítségével.
- **6. hét: Backend infrastruktúra.** A Python FastAPI szerver alapjainak lerakása, a relációs adatbázis (SQLAlchemy) és a modellek implementálása.
- **7. hét: Autentikáció és biztonság.** A JWT (JSON Web Token) token-generálás és az OAuth2.0 Google bejelentkezés integrálása a biztonságos session-kezelés érdekében.
- **8. hét: Rendszerintegráció.** A beérkező vélemények vektorizálását és Qdrant feltöltését végző adatfeldolgozó (ETL) pipeline megírása, a kontextus keresés és az LLM összekötése.
- **9. hét: Frontend fejlesztés (UI/UX).** A felhasználói felület kialakítása HTML, Tailwind CSS és Vanilla JavaScript technológiákkal.
- **10. hét: API és Kliens összekötése.** A frontend aszinkron fetch hívásainak összekötése a FastAPI végpontokkal, a hibakezelések (Error Handling) és a CORS (Cross-Origin Resource Sharing) szabályok finomhangolása.
- **11. hét: Cloud Deployment és tesztelés.** A rendszer élesítése a Render.com felhőszolgáltatón. A felmerülő port-konfigurációs és konfiguráció-betöltési hibák diagnosztizálása és javítása.
- **12. hét: Dokumentáció és prezentáció.** A projekt technikai és elméleti eredményeinek összefoglalása, a hivatalos beszámoló és a védéshez szükséges prezentációs anyagok elkészítése.

4. Rendszerarchitektúra és tervezés

4.1 Magas szintű rendszerfelépítés (Kliens-Szerver modell)

A ReviewAgent szoftverarchitektúrája a klasszikus, többretegű (N-tier) elosztott rendszerek mintáját követi, kiegészítve a modern felhőalapú mikroszolgáltatások (API-k) integrációjával. A rendszer fizikailag és logikailag is két fő komponensre válik szét: a böngészőben futó kliensoldalra és a felhőben futó kiszolgálóra.

A két réteg közötti kommunikáció kizárólag szabványosított, állapotmentes REST API végpontokon keresztül történik, JSON adatformátumban. Ez a szeparáció biztosítja a rendszer robusztusságát: a frontend mentesül a nehéz számítási feladatoktól, a backend pedig dedikáltan az üzleti logika végrehajtására, az adatbázis-kezelésre és a mesterséges intelligencia modellek felé történő hálózati kommunikációra fókuszálhat.

4.2 Adatmodellek és perzisztencia-réteg

A rendszer adattárolási stratégiája hibrid megközelítést alkalmaz. Az adatok természetétől függően két, egymástól független adatbázist használunk, amelyeket az üzleti logika szinkronizál:

1. **Relációs struktúra (PostgreSQL):** Itt tároljuk a szigorúan strukturált, tranzakcionális adatokat. Az adatbázis sémáját SQLAlchemy ORM segítségével definiáltam. A főbb entitások:
 - User: A regisztrált felhasználók adatai (email, titkosított jelszó, Google OAuth metaadatok).
 - Business: A felhasználóhoz tartozó KKV profilok, amelyek tartalmazzák az egyedi stílusirányelveket (Style Guideline) és a cég specifikus adatait.
 - Review: A beérkezett Google-értékelések strukturált másolata (szerző, dátum, csillagok, generált válasz állapota).
2. **Vektortér struktúra (Qdrant):** Itt tároljuk a véleményekből és a jóváhagyott válaszokból képzett matematikai vektorokat. Minden beszűrt vektorhoz csatolunk egy Payload (metaadat) objektumot, amely tartalmazza az eredeti szöveget és a relációs adatbázisból származó business_id azonosítót.

4.3 A RAG és Agentic AI Pipeline logikai folyamata

A szoftver legkomplexebb modulja a válaszgeneráló AI ágens. Amikor a rendszerbe egy új, megválaszolatlan ügyfélértékelés érkezik, az alábbi szigorúan szekvenciális, autonóm döntéshozatali (Agentic) folyamat indul el:

1. **Vektorizálás (Embedding):** A backend az új értékelés szövegét elküldi a gemini-embedding-001 API-nak, amely visszaad egy 3072 dimenziós lebegőpontos vektort.

2. **Kontextus-keresés (Retrieval):** A rendszer a generált vektorral egy koszinusz-hasonlósági keresést indít a Qdrant adatbázis felé. A keresés során a szoftver a Qdrant új, `query_points` API-ját használja. Kritikus lépés a szűrés: a keresést egy `FieldCondition` segítségével kizárólag a lekérdező cég `business_id`-jára korlátozzuk, így a rendszer csak a cég saját korábbi, releváns válaszait kapja vissza (Top-K találat).
3. **Prompt injekció (Augmentation):** Az ágens összeállít egy dinamikus rendszer-utasítást (System Prompt). Ebbe beágyazza a cég `Style Guideline`-jét, valamint az előző lépésben visszakapott korábbi, sikeres válaszokat mint kontextust.
4. **Generálás (Generation):** A gazdagított promptot a rendszer továbbítja a `gemini-2.5-flash` modellnek. A modell a kapott tényadatok és stílusminták alapján egy egyedi, hallucinációmentes választervezetet állít elő, amelyet a backend JSON formátumban visszaküld a kliensnek jóváhagyásra.

4.4 Biztonsági architektúra (Autentikáció és Autorizáció)

Mivel az alkalmazás szenzitív üzleti adatokat kezel, a biztonság a tervezés nulladik lépése volt. A bejelentkezési folyamat szabványosított Google OAuth2.0 protokollal, vagy saját hitelesítési végponton keresztül történik.

A sikeres hitelesítést követően a backend egy kriptográfiailag aláírt **JWT (JSON Web Token)** azonosítót állít elő. A tokenek kezelése során a legmagasabb biztonsági elveket követtem: a JWT token nem a sebezhető kliensoldali `localStorage`-ban tárolom, hanem egy `HttpOnly` és `Secure` flaggel ellátott süti (Cookie) csomagolva küldöm vissza. Ez a mechanizmus a böngésző szintjén teszi lehetetlenné, hogy kártékony JavaScript kódok (XSS támadás) hozzáférjenek a tokenhez.

Minden egyes API hívás (vélemények lekérése, generálás) során a backend middleware-je ellenőrzi a token, kibontja belőle a felhasználó azonosítóját, és a relációs adatbázis lekérdezéseihez ezt használja. Ezzel a megoldással a rendszer teljesen védett az Insecure Direct Object Reference (IDOR) sebezhetőségek ellen, garantálva a multi-tenant adatbiztonságot.

5. Félév során megvalósított rendszer, architektúra

5.1. Komponens-alapú architektúra és moduláris felépítés

A ReviewAgent backend rétege szigorúan követi a tiszta architektúra és a felelősség szétválasztásának (Separation of Concerns) elveit. A forráskód logikailag és strukturálisan jól elkülöníthető modulokra bomlik, amelyek dedikált feladatokért felelnek:

- **main.py (Alkalmazás belépési pontja és API réteg):** Ez a modul inicializálja a FastAPI alkalmazást, konfigurálja a CORS (Cross-Origin Resource Sharing) szabályzatot, beolvassa a környezeti változókat, és definiálja a HTTP végpontokat, amik a következők: /auth/register, /auth/login, /generate, /reviews, /analytics. Itt történik a beérkező HTTP kérések fogadása és a strukturált JSON válaszok kiküldése.
- **models.py (Adatperzisztencia és ORM réteg):** Ez a fájl tartalmazza a relációs adatbázis sémáját leíró SQLAlchemy osztályokat. Ebben a modulban definiáltam az entitások közötti kapcsolatokat vagyis egy felhasználóhoz több vállalkozás, egy vállalkozáshoz több vélemény tartozhat.
- **schemas.py / schemas (Adatvalidációs réteg):** A Pydantic könyvtárra épülő adatmodellek, amelyek a bejövő és kimenő adatok szigorú típusellenőrzéséért és validációjáért felelnek. Az objektumok adatbázis-független adatátvitelre képesek – DTO.
- **db_feltoltes.py (ETL és adatbetöltő pipeline):** Egy önállóan is futtatható vagy a háttérfolyamatok által meghívható modul, amely a strukturálatlan Google véleményeket (CSV/API forrásból) képes beolvasni, a Gemini-embedding modellen keresztül vektorizálni, és strukturált metaadatokkal payload-dal kiegészítve feltölteni a Qdrant vektoradatbázisba.

5.2. Osztálydiagram és relációs adatstruktúra

A rendszer statikus felépítését, az adatbázis-modellek és az API kéréseket validáló Pydantic sémák közötti strukturális kapcsolatot az alábbi statikus osztálydiagram szemlélteti. Az entitások között logikahierarchia és függőségek lének fel.

1. **User entitás:** A rendszer belépési pontja. Egyedi azonosítóval (id), egyedi e-mail címmel és hashelt jelszóval rendelkezik. Egy-a-többhöz (1:N) kapcsolatban áll a Business entitással, ami

azt jelenti, hogy egy felhasználó több cég vagy telephely profilját is kezelheti a SaaS rendszeren belül.

2. **Business entitás:** Tárolja a KKV specifikus adatait (név, kategória, leírás). Itt kapott helyet a `style_guideline` szöveges mező is, amely az ágens működését szabályozza. Egy-a-többhöz (1:N) kapcsolatot tart fenn a Review entitással.
3. **Review entitás:** A Google Business Profile-ből származó értékelések leképezése. Tartalmazza a szerző nevét, a csillagok számát, a szöveges értékelést, a beérkezés dátumát, valamint a rendszer által generált, vagy a felhasználó által módosított és jóváhagyott válasz szövegét.

A szoftverfejlesztés során kiemelt figyelmet fordítottam arra, hogy az adatbázis-modellek attribútumai szigorú típusossággal rendelkezzenek, minimalizálva a futásidejű típuskonverziós hibákat.

5.3. Dinamikus működés és szekvenciadiagram

A rendszer dinamikus működését, az aszinkron kliensoldali kérések és a külső MI (Gemini, Qdrant) felhőszolgáltatások közötti adatáramlást a válaszgenerálási folyamat során az alábbi szekvenciadiagram mutatja be.

1. **Kliensoldali kérés:** A felhasználó a Tailwind alapú dashboardon rákattint a *"Válasz generálása"* gombra. A frontend egy aszinkron POST kérést indít a backend `/generate` végpontjára, elküldve a vélemény azonosítóját és a cég egyedi azonosítóját.
2. **Hitelesítés és autorizáció:** A FastAPI middleware-je ellenőrzi a HTTP-only sütiből kiolvasott JWT tokenet. Amennyiben a token valid, ellenőrzi, hogy a bejelentkezett felhasználónak van-e jogosultsága az adott `business_id` adataihoz.
3. **Vektorizálás (Embedding):** A backend lekéri az adatbázisból az új vélemény szövegét, majd meghívja a google-genai SDK segítségével a `gemini-embedding-001` modellt. Az API visszaküldi a szöveg matematikai reprezentációját képező vektort.
4. **Kontextus-visszakeresés (Retrieval):** A backend a kapott vektorral lekérdezést intéz a Qdrant Cloud felé a `q_client.query_points()` metóduson keresztül. A kérés tartalmaz egy szigorú szűrőfeltételt (`payload["business_id"] == request.business_id`), így a Qdrant kizárólag az adott vállalkozás múltbéli, sikeres válaszait adja vissza.

5. **Dinamikus kontextus-bővítés (Augmentation):** A backend összefűzi a cég alapadatait, a Qdrantból visszakapott legrelevánsabb korábbi válaszokat, valamint a cég stílusirányelvét egy összetett Prompt-tá.
6. **Szöveggenerálás (Generation):** Az így előkészített, tényadatokkal alátámasztott promptot a rendszer elküldi a gemini-2.5-flash modellnek. A modell másodperceken belül legenerálja a hallucinációmentes, nyelvtanilag tökéletes választ.
7. **Válasz és perzisztencia:** A backend a generált választervezetet elmenti a PostgreSQL adatbázisba ideiglenes állapotúként, majd JSON formátumban visszaküldi a kliensoldalnak, ahol az interaktív módon megjelenik a felhasználó előtt.

5.4. Adatszeperáció és gyűjtemény-struktúra a vektoradatbázisban

Mivel a rendszer több cég adatait kezeli egyetlen közös felhőalapú Qdrant klaszterben, az adatszeperációt logikai szinten, a pontokhoz csatolt Payload struktúrával valósítottam meg. Egy Qdrant-pont az alábbi mérnöki struktúrával kerül tárolásra:

JSON

```
{
  "id": "uuid-v4-azonosito",
  "vector": [0.0125, -0.0432, ..., 0.0091],
  "payload": {
    "business_id": 1,
    "review_text": "Kiváló kiszolgálás, csak a parkolás nehéz.",
    "approved_reply": "Köszönjük a visszajelzést! Örömmel halljuk, hogy elégedett volt a kiszolgálással. A parkolási lehetőségek bővítésén folyamatosan dolgozunk."
  }
}
```

A keresés indításakor a Qdrant belső indexelő motorja először végrehajtja a payload-alapú szűrést, ami egy rendkívül gyors, $O(1)$ komplexitású hash-alapú szűrés a relációs kulcson, és a matematikai koszinusz-hasonlósági számítást már csak ezen a szűrt részhalmazon futtatja le. Ez a megközelítés nemcsak a legmagasabb szintű adatbiztonságot garantálja, de radikálisan csökkenti a keresés számítási igényét is, biztosítva a rendszer horizontális skálázhatóságát.

6. Megvalósítás részletei, tapasztalatok

6.1. Az adatbázis feltöltése és a RAG pipeline előkészítése (ETL folyamat)

A rendszer működésének alapfeltétele a korábbi ügyfélértékelések strukturált betöltése a Qdrant vektoradatbázisba. Ezt az Extract, Transform, Load (ETL) folyamatot a `db_feltoltes.py` szkript végzi el. A megvalósítás során aszinkron adatfeldolgozást alkalmaztam. A szkript beolvassa a nyers adatokat, a Google Gemini `gemini-embedding-001` modelljével legenerálja a 3072 dimenziós vektorokat, majd egy szigorúan típusos Payload objektum kíséretében feltölti azokat a felhőbe.

Kiemelt figyelmet igényelt a feltöltési struktúra kialakítása, hogy a későbbi szűrések (Multi-tenancy) hibátlanul lefuthessenek:

```
response = genai.embed_content(
    model="models/gemini-embedding-001",
    content=review_text
)
embedding = response['embedding']

# Payload összeállítása az adatszeparációhoz és kontextushoz
payload = {
    "business_id": business_id,
    "review_text": review_text,
    "rating": rating,
    "reply_text": reply_text
}

# Rekord beszúrása a Qdrant adatbázisba UUID-vel
q_client.upsert(
    collection_name=COLLECTION_NAME,
    points=[
        PointStruct(
            id=str(uuid.uuid4()),
            vector=embedding,
            payload=payload
        )
    ]
)
```

6.1. Ábra: Vektorizálás és feltöltés

6.2. A Qdrant API verzióváltása és a keresési logika implementációja

A fejlesztés során egy váratlan technológiai kihívást jelentett a Qdrant Python SDK v1.16+ verzióváltása. Míg a lokális, régebbi fejlesztői környezetben a `q_client.search()` metódus felelt a koszinusz-hasonlósági lekérdezésekért, a felhőalapú Render telepítés során a legfrissebb könyvtár már megszüntette ezt a funkciót.

A problémát a szoftver kódbázisának azonnali refaktorálásával oldottam meg a `main.py` fájlban. Átálltam a modernebb `query_points()` API használatára, amely robusztusabb szűrési paraméterezést tesz lehetővé. Ez a kódrészlet felel a RAG architektúra legfontosabb lépéséért, a releváns múltbéli esetek ügyfél-specifikus (`business_id`) visszakereséséért:

```
query_response = q_client.query_points(  
    collection_name=COLLECTION_NAME,  
    query=query_vector,  
    query_filter=q_models.Filter(  
        must=[  
            q_models.FieldCondition(  
                key="business_id",  
                match=q_models.MatchValue(value=request.business_id),  
            )  
        ],  
    ),  
    limit=3  
)  
search_result = query_response.points
```

6.2. Ábra: biztonságos keresés

6.3. Kísérletek: Lokális (Ollama) vs. Felhőalapú (Gemini) LLM

A félév első harmadában a mesterséges intelligencia modellek lokális (On-Premise) futtatásával kísérleteztem. A cél egy nyílt forráskódú modell: Gemma29B futtatása volt Ollama környezetben. A méréseket egy standard fejlesztői munkaállomáson, 16GB RAM-on végeztem, 100 darab, átlagosan 50-80 szavas magyar nyelvű prompt generálásával. Az eredményeket a felhőalapú Gemini 1.5 Flash modell API-n keresztül mért eredményeivel vetettem össze.

Metrika / Paraméter	Lokális (Ollama - Gemma29B)	Felhő (Gemini 1.5 Flash API)
Átlagos Válaszidő (TTFT)	~42,7 másodperc	< 2,5 másodperc
Magyar nyelvhelyesség	Szintaktikai hibák, nem magyaros szófordulatok	Kiváló natív nyelvi szint
Hardverigény (RAM / VRAM)	Magas (minimum 16GB, GPU dedikáció)	Alacsony
RAG Kontextus betartása	Gyakran hallucinált a megadott adatokon kívül	Szigorúan követte a JSON stílusirányelvet

A mérések egyértelműen bizonyították, hogy a KKV szektornak szánt SaaS termékek esetében a lokális modellek futtatása jelenleg nem skálázható. A közel 43 másodperces válaszidő elfogadhatatlan egy modern webes felületen, így a végleges architektúra a Gemini API mellett köteleződött el.

6.4. Fejlesztési tapasztalatok és megoldott mérnöki kihívások

A projekt implementációja során több olyan rendszer- és hálózatszintű problémával szembesültem, amelyek megoldása jelentős mérnöki tapasztalatot igényelt, alább látható a 3 legfejfájósabb kihívás.

1. Hardveres és driver szintű konfliktusok (WSL2 és Docker): A lokális LLM kísérletek során a Docker és a Windows Subsystem for Linux (WSL2) réteg között PCI/Chipset driver ütközések léptek fel a dedikált memóriakezelés során. Ez rendszeres fagyásokhoz vezetett a modell betöltésekor. A tapasztalat azt mutatta, hogy az AI fejlesztések esetében a natív Unix-alapú környezetek, vagy a teljes egészében menedzselt felhőszolgáltatások alkalmazása elkerülhetetlen.

2. Külső API korlátozások (Google Quota 0 probléma): A Google Business Profile (Cégem) valós idejű vélemény-szinkronizációjának integrálása során az API hitelesítés sikeres volt, azonban a Google belső kvótaszabályozása miatt a hivatalos vállalati regisztráció lezárultáig a végpontok éles adatokkal nem voltak lekérdezhetőek. Ezt a blokkoló tényezőt egy robusztus Workaround implementálásával hidaltam át: a valós API hívásokat egy saját fejlesztésű ETL szimulációs pipeline-nal helyettesítettem, amely CSV alapú, lekapart adatbázisból tölti fel a rendszert.

3. Felhő-telepítési (Deployment) anomáliák a Render platformon: A rendszer élesítése során a Render port-scannere "Port scan timeout reached" hibát dobott. Ennek oka a Python környezet függőségeinek forrásból történő, időigényes újrafordítása volt az ingyenes szervereken. A megoldást az automatikus környezetfelismerés felülbírálata (a PYTHON_VERSION környezeti változó fix 3.11/3.12 verzióra állítása), az explicit port-binding (10000-es port megadása), valamint a Build Cache teljes törlése jelentette, amely drasztikusan lerövidítette a telepítési időt.

7. Képernyőképek, fotók az elkészült megoldásról

A szoftver fejlesztése során a robusztus backend architektúra mellett kiemelt szerepet kapott a felhasználói élmény (UX) és az ergonómia. A kliensoldali felület Tailwind CSS keretrendszerrel készült, amely modern, reszponzív, és a SaaS iparági sztenderdeknek megfelelő sötét/világos tónusú megjelenést biztosít.

7.1. A felhasználói felület: Frontend nézetei

A végfelhasználók az alábbi interaktív felületeken keresztül kommunikálnak a mesterséges intelligenciával és tekintik át az üzleti statisztikákat.

- 7.1. ábra: *A ReviewAgent központi irányítópultja az automatikusan generált, kontextus-vezérelt választervezettel.*
- 7.2. ábra: *Vizuális adatmegjelenítés a Chart.js könyvtár segítségével az analitikai nézeten.*
- 7.3. ábra: *A rendszer reszponzív, JWT-alapú hitelesítési felülete (Login).*

7.2. Kiemelt architekturális kódrészletek a Backend-ből

A vizuális felület mögött meghúzódó üzleti logikát és a komplex RAG folyamatot az alábbi kritikus kódszegmensek biztosítják.

- 7.4. ábra: *A RAG pipeline legfontosabb lépése: a Qdrant vektoradatbázis payload-alapú, multi-tenant szűrése a koszinusz-hasonlósági keresés során.*
- 7.5. ábra: *Az autonóm ágens rendszer-utasításának (System Prompt) dinamikus összeállítása és a Gemini 2.5 Flash API hívása.*
- 7.6. ábra: *A biztonságos, kliensoldali támadások (XSS) ellen védett HttpOnly session süti beállítása a FastAPI szerveren.*

8. Továbbfejlesztési lehetőségek

A ReviewAgent jelenlegi verziója egy stabil, felhőben futó, funkcionálisan teljes prototípus, az ilyen fázisban lévő projekteket MVP – Minimum Viable Product – nak nevezzük, amely sikeresen bizonyítja a RAG és az Agentic AI technológiák létjogosultságát a KKV szektor ezen részén. A projekt továbbfejlesztése – amely egy jövőbeli Diplomaterv alapját is fogja alkotni – az alábbi kritikus mérnöki és üzleti fókuszterületekre oszlik.

8.1. Hivatalos Google App Verification és valós idejű szinkronizáció

A jelenlegi félév fejlesztési szakaszában a Google Business Profile API szigorú biztonsági és kvótaszabályozása miatt a rendszer ETL szimulációs eljárásokkal töltötte be a vektorteret. A következő időszak legfontosabb feladata a szoftver teljes körű hitelesítési folyamatának a Google Cloud OAuth App Verification hivatalos lebonyolítása. Ennek sikeres lezárásával a szoftver képes lesz elhagyni az aszinkron kötegelt adatfeldolgozást, és a Google Webhookjain keresztül valós idejű, kétirányú szinkronizációt valósíthat meg, közvetlenül publikálva a jóváhagyott AI válaszokat.

8.2. A felhasználói felület (UI/UX) kiterjesztése és modernizálása

Bár a jelenlegi Tailwind CSS alapú natív kliensoldal gyors és biztonságos, a funkciók bővülésével indokoltá válik a felület kiterjesztése. A tervek között szerepel a frontend migrációja egy modern, komponens-alapú keretrendszerbe (React.js). A UI továbbfejlesztése magában foglalja a WebSocket-alapú valós idejű értesítések (értesítési csengő új vélemény beérkezésekor) implementálását, a sötét/világos mód (Dark/Light mode) natív támogatását, valamint az analitikai modul (Chart.js) interaktivitásának növelését részletesebb szűrőkkel.

8.3. Multi-platform integráció és az MCP bevezetése

A piaci relevanciát jelentősen növelné, ha a rendszer nem korlátozódna kizárólag a Google értékelésekre. A következő félév feladatai közé tartozna a szoftverarchitektúra olyan irányú általánosítása, amely lehetővé teszi további véleményplatformok, mint például a Facebook Reviews, Trustpilot, vagy TripAdvisor API szintű bekötését. Emellett technológiai innovációt jelentene a Model Context Protocol (MCP) integrációja, amely szabványosított hidat képezne a helyi AI ágens és a cégek külső rendszerei (CRM szoftverek) között, így a generált válaszok még specifikusabb kontextusból meríthetnének.

8.4. SaaS skálázódás és autonóm publikálási szintek

Üzleti szempontból a szoftver teljes értékű SaaS termék formálása a cél. Ez technológiai oldalról megköveteli egy automatizált előfizetési és számlázási modul bekötését. További fejlesztési lehetőség a publikálási folyamat finomhangolása: egy olyan konfidencia-küszöbérték (Confidence Score) bevezetése, amely felett a rendszer (az üzletvezető engedélyével) emberi felülbírálat nélkül, teljesen autonóm módon is képes válaszolni a pozitív, egyszerűbb értékelésekre, ezzel maximalizálva a reakcióidőt és a SEO előnyöket.